Messaging: merge, load rerun and the state cache

Work report for the **messaging** worktree, 10 October 2026. Branch `feat/messaging`, commits `9845c527` to `5721e1d9`.

1. Summary

- **Branch brought up to date.** The edge-trust commit from `announce-jo-branch` was merged in; the messaging suite passes with it.
- **The open measurement was taken.** Under the baseline load, the race fixes of 1 October lose no post: 1,599 of 1,599 acknowledged over two runs, zero errors.
- **The state cache is on.** Step 2 of the tuning order: the command side keeps conversation state in an LRU cache. On 200-message conversations a post went from 3.5 s to 78 ms, and from half of them lost to none.
- **The next bottleneck is named.** The read-model projection group reaches its capacity at about 22 events a second per partition; steps 3 and 6 of the tuning order address it.
- Five commits today, all 34 tests green, the branch level with `announce-jo-branch` and ahead of it; nothing pushed.

2. Merge of announce-jo-branch

The integration branch had gained the edge-trust change: backends now refuse requests without a valid `X-Service-Token`, and the messaging end-to-end test turns that check on. The merge had one conflict, an import block in the end-to-end test, and one adaptation: the REST burst of the racing-posts test had to sign its requests like an edge, or the token check would answer 401.

What came in	Effect on messaging
`ServiceTokenWebFilter` in `cqrs-reactive`	Requests and WebSocket handshakes without a valid token are refused; the test profile keeps it off, the end-to-end test turns it on
`Principals` comment and `application.yml`	Documents that the headers are trusted only behind an edge; the announce lookup now sends the token
New end-to-end case	A direct request with identity headers but no token is refused; the suite grew to 33 tests

3. Load rerun after the race fixes

The retry for racing posts was verified by tests on 1 October; its stated measure was a zero error count under the baseline load. Two 60-second runs of the load driver were taken against one node, 20 conversations and 40 sockets each posting every 3 s. The host was not idle: a video call held about two cores, and during run 1 another worktree ran three test suites.

	Baseline, 29 Sep	Run 1, busy host	Run 2, quieter
posts sent / acknowledged	800 / 788	799 / 799	800 / 800
error frames	12 (1.5 %)	0	0
retries	none existed	316	45
ack latency p50 / p95	66 / 131 ms	1144 / 4152 ms	142 / 350 ms
read-model handler p50	41 ms	103 ms	73 ms

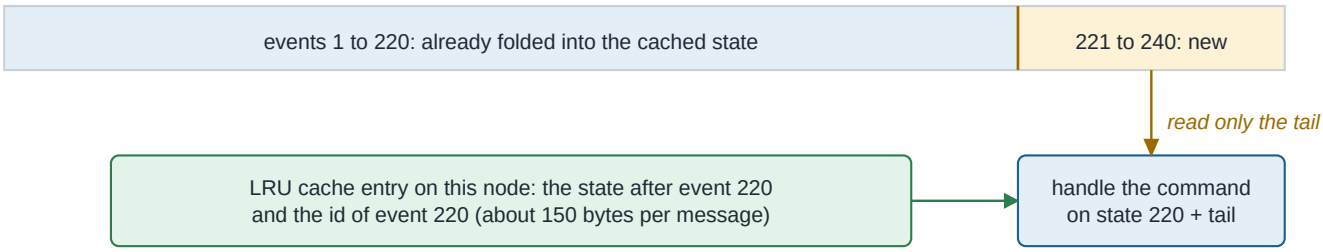
	Baseline, 29 Sep	Run 1, busy host	Run 2, quieter
read-model lag p50	84 ms	23 s	6.8 s

- **The defect is closed.** No post was lost in either run and the read model held every message. Under the busy host 316 posts collided at least once; all were absorbed, none reached the retry bound.
- **Latencies are an upper bound, not a new baseline.** The host conditions differ from the baseline's, which were not recorded.
- **The read-model group ran at its limit.** At 73 ms per event one partition absorbs about 14 events a second against 13 posts a second, so the lag grew to 7 s and cleared after the run.

4. The state cache

Why. OpenCQRS rebuilds the state of a conversation from its events on every command. The service ran with no cache, so a post into a conversation with N messages read and folded N events first. The baseline could not show this cost because the driver only used fresh conversations.

Events of one conversation in EventSourcingDB



Without the cache the router reads and folds all 240 events on every command.

Figure 1. The cache holds the state and the id of the last event it saw; the router reads only what came after. A stale entry on another node is read forward, never served as is.

What changed

- `opencqrs.command-handling.cache.type=in_memory` with `capacity` 1000 in the main and the test configuration. One entry grows by about 150 bytes per message, so the capacity bounds memory at roughly 150 KB per 1,000-message conversation.
- The end-to-end test pins the cache type on the command router, so a configuration slip cannot silently fall back to full replay.
- The load driver takes `-Dload.prefill=N` and posts N messages into every conversation before the sockets start, in parallel across conversations and sequentially within each.

Measurement

Two runs back to back on the same node and host, prefill 200, so every post worked on a history of 200 to 240 events. Run A started the node with the cache set to none, run B with the new default.

	A: cache off	B: cache on
prefill of 4,000 messages over REST	323 s	211 s
posts sent / acknowledged within the run	798 / 369	800 / 800
error frames	79	0
retries	271	37
ack latency p50 / p95 / max	8.7 / 15.8 / 23.4 s	74 / 295 / 554 ms

	A: cache off	B: cache on
command latency p50, successful posts	3.5 s	78 ms
command latency p50, failed posts	14.0 s	none
read-model handler p50	82 ms	45 ms
read-model lag p50	0.15 s	54 s
heap old generation after the run	187 MB	177 MB

- **Without the cache a long conversation is unusable at this load.** Each post replayed about 220 events and took 3.5 s; half the posts were still unanswered when the run ended, and 79 failed outright because four attempts of 3.5 s each reach the retry bound at the 14 s the error timer shows.
- **With the cache the history costs nothing.** 78 ms per post is the figure of a fresh conversation. Every post was acknowledged and 37 collisions were absorbed. Memory did not grow.
- **The read-model group is now the bottleneck.** 4,800 events arrived in about four and a half minutes against about 22 events a second per partition, so the lag reached 54 s before clearing. The command side is no longer the limit.

5. Housekeeping and hazards

- **Checkpoints before a node starts.** After a test run the three checkpoint rows hold event ids of the test's container. They were set to the local store's last conversation event id before each node start, so nothing old was replayed onto the public topics. The recipe is in `doc/messaging-context.md` section 7.
- **Sibling modules installed from this tree** into the local Maven repository before the node ran, so the node used this tree's versions rather than another worktree's.
- **Load rows removed.** All 40 load conversations of the cache runs, with 9,209 messages, were deleted from the read model afterwards; the morning's 40 conversations likewise.
- **Concurrent test runs.** The suite was only started while no other worktree was testing, after the collision seen on 1 October.

6. Commits and state

Commit	Content
`9845c527`	Work report of 1 and 2 October (PDF)
`42bb9b65`	Merge of announce-jo-branch; import conflict resolved, REST burst signed
`954c4b16`	Load rerun recorded in doc/messaging-architecture.md section 7a
`1e5bd516`	State cache on, end-to-end pin, driver prefill, A/B in section 7b
`5721e1d9`	Second merge of announce-jo-branch (the announce-kinds repair: SQL and announce-management only)

- **Branch:** `feat/messaging` is seven commits ahead of `announce-jo-branch` and none behind. Nothing pushed.
- **Tests:** 34 in the messaging suite, all passing on Testcontainers plus the live MySQL.
- **Next decision:** merge `feat/messaging` into `announce-jo-branch` from the main checkout; the branch is checked out there, so it cannot be done from this worktree.
- **Next tuning steps:** 3 and 6 of the order in `doc/messaging-architecture.md`: one processor set per group across nodes, then more read-side partitions and a larger connection pool, measured by the read-model lag.